

Contents

A small language model that reports its own embedding coordinates	1
Summary	1
1. The question	2
2. Setup	2
3. Measures	3
4. Language modelling	4
5. Accuracy and follow, all runs	4
6. Why the joint runs generalize worse	5
6.1 Rare tokens' embedding vectors	5
6.2 Rescaling test	5
6.3 Interpretation and an open alternative	6
7. Response to the norm of $E[t]$, and LayerNorm	6
8. Editing an embedding vector inside the model	7
9. A linear map from the LM-only model's internal state	8
10. What this does and does not show	8
11. Checks	9

A small language model that reports its own embedding coordinates

Summary

Question. Can a language model learn to report the values of its own weights, with answers computed from the current weights rather than memorized? The weights asked about here are the token embeddings.

Model and task. A small GPT-style transformer (4 blocks, 1.35 million parameters; the internal state at each position, passed from block to block, is a vector in $\mathbb{R}^{128}$) is trained on TinyStories, a corpus of simple children's stories, for 15,000 steps. Input and output embeddings are tied: each of the 4,096 tokens t has one embedding vector $E[t] \in \mathbb{R}^{128}$, used both to read text and to score the next token. Besides predicting the next token, the model is trained on a self-report task: given the three input symbols [QUERY, COORD_ i , t], it must output the number $E[t, i]$, the current value of coordinate i of t 's embedding vector, through a linear output head. Questions are asked about 3,072 randomly chosen tokens (asked-about tokens). The other 1,024 (never-asked tokens) are used only to test whether the answers generalize; all but 47 of them occur in the text the model is trained on.

Runs. - LM-only (seed 0): next-token prediction only. - Joint (seeds 0 and 1): next-token prediction plus the self-report task; both tasks update all parameters, including the token embedding vectors. - Control (seed 0): as joint seed 0, except that the self-report task cannot change the token embedding vectors (its gradient is stopped there). The token embedding vectors are then shaped by next-token prediction alone.

Measures. - Centred R^2 : $1 - \Sigma(\text{answer} - \text{target})^2 / \Sigma(\text{target} - \text{that coordinate's mean over the tokens tested})^2$. It is 1 for exact answers and 0 for always answering the coordinate's mean. - Follow: J is the 128×128 matrix of derivatives of the 128 answers about t with respect to the 128 coordinates of $E[t]$. Follow is $\text{tr}(J)/128$, averaged over tokens: the average amount by which the answer about coordinate i moves when coordinate i of $E[t]$ moves by one unit. It is 1 if each answer moves exactly with its coordinate, and 0 if the answers do not respond to $E[t]$.

run	validation loss (nats per token)	centred R^2 , asked-about / never-asked tokens	follow, asked-about / never-asked tokens
LM-only, seed 0	2.146		
joint, seed 0	2.248	0.976 / 0.680	0.851 / 0.751
joint, seed 1	2.290	0.955 / 0.504	0.745 / 0.666
control, seed 0	2.290	0.982 / 0.982	0.871 / 0.868

Findings. 1. The self-report task raises the validation loss by 0.10 nats per token (joint, seed 0) or 0.14 (control), compared with LM-only. There is no LM-only run with seed 1. 2. In the control, answers about never-asked tokens are as accurate as answers about asked-about tokens. 3. In the joint runs, answers about never-asked tokens are much less accurate. The evidence for the cause (seed 0; seed 1 agrees): - In LM-only, rare tokens (fewer than 100 occurrences in the training text) end with embedding vectors of norm about 4.7, against 1.4 to 1.7 for the others. - In joint seed 0, rare asked-about tokens end with norm 1.53, rare never-asked tokens with 3.95. Answers about the latter respond little to their vectors (follow 0.34) and are inaccurate (rescaling score, defined in section 6.2: -1.34 , where 1 means exact). Rescaled to norm 1.41 (the mean norm of frequent never-asked tokens), keeping their direction, they score 0.96. - In the control, rare tokens end with large norms whether asked about or not (4.11 and 4.13), so vectors with large norms are among the questions in training. Answers about rare never-asked tokens score 0.99, although their follow (0.62) is lower than for other tokens (0.90).

Interpretation: when it can, the self-report task changes the asked-about embedding vectors so that the answers work for them; the never-asked vectors keep the shape next-token prediction gave them, and the answers do not carry over to them. 4. Answers are never exact. For a small change δ of $E[t]$, the 128 answers also move by 0.35 to 0.48 of $|\delta|$ in ways other than following δ . They respond less to a change of the norm of $E[t]$ than to changes of its direction (section 7).

1. The question

Can a language model answer questions about its own weights, with answers that come from the weights themselves? The test, as in `../self-report-embeddings/`: change a weight and see whether the answer changes with it. There, a toy model with random embedding vectors and no language modelling learned to answer almost exactly (follow 0.997 to 1.005) once it was asked about 3,072 distinct embedding vectors. Here the embedding vectors are shaped by next-token prediction and shared with the output, and the model has positional embeddings and LayerNorm.

2. Setup

Data. TinyStories (Hugging Face, roneneldan/TinyStories): training shard 0 of 4 (529,930 stories, 122 million tokens; the training text) and the validation set (21,990 stories, 4.9 million tokens). Tokenizer: byte-level BPE with 4,096 tokens, including an end-of-story token, trained on the training text.

Model. A GPT-style decoder: 4 blocks (each: LayerNorm, causal self-attention with 4 heads, LayerNorm, feed-forward layer of width 512 with GELU), internal states in $\mathbb{R}^{128}$, learned positional vectors $P[0], \dots, P[127]$, context of 128 tokens. Each block applies LayerNorm to its input before using it: subtract the mean of the 128 coordinates, divide by their standard deviation, then apply a learned scale and shift per coordinate. 1,350,657 parameters in all.

Embedding table. One vector in $\mathbb{R}^{128}$ for each of the 4,096 tokens and for each of 129 extra symbols (QUERY, COORD_0, ..., COORD_127). The input to the first block at a position is the embedding vector of the symbol there plus the positional vector of that position. At the output, the score of each possible next token is the inner product of the last internal state (after a final

LayerNorm) with that token’s embedding vector (tied embeddings). The extra symbols are never predicted.

Self-report task. A question (t, i) is the sequence [QUERY, COORD_i, t]; t is at position 2 (counting from 0). The answer is $w \cdot h + b$, where h is the internal state at position 2 after the last block and before the final LayerNorm, and $w \in \mathbb{R}^{128}$ and b are the weights of the output head. The target is $E[t, i]$, the current value. The task loss on a batch of 256 questions is $1 - R^2$ of the batch, where this R^2 uses a single mean over the batch (uncentred R^2 , unlike the centred R^2 used for measurement). No gradient flows through the targets.

Asked-about and never-asked tokens. A random three quarters of the 4,096 tokens (3,072) are asked about during training; the other 1,024 are never asked about. The choice depends on the seed. Both kinds occur in the training text, except 88 asked-about and 47 never-asked tokens (seed 0) that never occur in it.

Training, all runs. 15,000 steps of AdamW (learning rate $2 \cdot 10^{-3}$ after 200 warm-up steps, cosine decay to $2 \cdot 10^{-4}$; weight decay 0.1 on the block and output-head matrices only; the norm of the combined gradient clipped at 1). Each step uses 32 windows of 128 tokens from the training text (61 million tokens seen in all), and, in the joint runs and the control, 256 random questions about asked-about tokens. Loss: next-token cross-entropy plus λ times the self-report loss, with loss coefficient $\lambda = 1$.

The four runs.

run	self-report task	gradient of the self-report loss reaches the token embedding vectors	seed
LM-only	no	(no self-report loss)	0
joint	yes	yes	0 and 1
control	yes	no (stopped at $E[t]$; it still trains the blocks, the output head, and the QUERY and COORD vectors)	0

Runs with the same seed start from the same weights and see the same text in the same order. Each run took 2.0 to 2.3 hours on 2 CPU threads.

3. Measures

All measures are taken after training, with all weights fixed.

- Validation loss: average of $-\log(\text{probability given to the actual next token})$ over 100×32 fixed windows of 128 tokens of the validation set, in nats per token. (Section 4 also uses the training logs, which used 20×32 windows.)
- Centred R^2 (defined in the summary), over all 128 questions about every token of the set measured.
- J, follow (defined in the summary). For a unit vector $u \in \mathbb{R}^{128}$, the gain along u is $u^T J u$: how much the answers move along u , per unit change of $E[t]$ along u . Follow is the average gain over the 128 coordinate axes.
- Other movement: for each token, $\|J - fI\|_F / \sqrt{128}$, where f is that token’s follow; averaged over tokens. It is the root-mean-square size, over random directions of a small change δ , of the part of the answers’ movement that is not $f \cdot \delta$, as a fraction of $|\delta|$. 0 for exact answers.
- Finite-change follow: add to $E[t]$ a vector δ in a random direction with norm 10% of $|E[t]|$; if the 128 answers change by Δ , the value is $\Delta \cdot \delta / |\delta|^2$.

- Token frequency: number of occurrences in the training text. Rare: fewer than 100; common: 100 to 9,999; frequent: 10,000 or more.

Follow, other movement and the gains use 512 random asked-about and 512 random never-asked tokens, unless stated otherwise.

4. Language modelling

Validation loss: LM-only 2.146; joint seed 0 2.248 (+0.102); joint seed 1 2.290; control 2.290 (+0.144).

Joint seed 0 against LM-only during training (training-log windows):

step	joint, seed 0	LM-only	difference
1,500	3.094	2.854	0.240
4,500	2.609	2.487	0.122
9,000	2.388	2.284	0.104
15,000	2.261	2.154	0.107

The difference shrinks during the first half of training and stays between 0.10 and 0.11 from step 7,500 on. LM-only reached joint seed 0’s final loss on these windows (2.261) at about step 9,700. This difference may come from the two tasks competing for model capacity, or from optimization: the two losses share Adam’s running statistics and one gradient-norm clip. This experiment does not tell the two causes apart.

Samples (prompt “Once upon a time”, temperature 0.8, three per run, in results/measure_<run>.json) are fluent children’s-story text in every run, with lapses in coherence in every run, LM-only included. For example, joint seed 0: “Once upon a time, there was a little girl named Lily. She loved to play with her toys and her toys. One day, she went to the park with her mommy. She saw the big red ball that was round and shiny.” and “The knight was always not sure to turn and be successful.”

5. Accuracy and follow, all runs

measure	joint, seed 0	joint, seed 1	control, seed 0
centred R ² , asked-about tokens	0.976	0.955	0.982
centred R ² , never-asked tokens	0.680	0.504	0.982
centred R ² , random vectors	0.813	0.699	0.821
follow, asked-about tokens (std over tokens)	0.851 (0.02)	0.745 (0.03)	0.871 (0.11)
follow, never-asked tokens (std over tokens)	0.751 (0.16)	0.666 (0.15)	0.868 (0.11)
finite-change follow, mean: asked-about / never-asked	0.848 / 0.748	0.742 / 0.661	0.866 / 0.861
other movement: asked-about / never-asked	0.41 / 0.40	0.48 / 0.46	0.35 / 0.35

Random vectors: 1,024 vectors drawn from a normal distribution with the mean and standard deviation of each coordinate over all 4,096 embedding vectors, put at position 2 in place of E[t]; question i asks for their coordinate i. LM-only is not in the table: its output head was never trained (its follow is -0.001).

Observations. - In the control, the answers are equally accurate for asked-about and never-asked tokens (0.982 and 0.982), and equally responsive (follow 0.871 and 0.868). - In the joint runs, the answers are much less accurate for never-asked tokens than for asked-about tokens (0.680 against 0.976; 0.504 against 0.955), and less responsive. - In all runs, follow is well below 1 and other movement is 0.35 to 0.48: the answers move partly with the coordinate asked about and partly in other ways. For comparison, the toy model of `../self-report-embeddings/` reached follow 0.997 to 1.005 and other movement 0.06 to 0.10. - The two joint seeds differ a lot (0.680 and 0.504); both are far below the control (0.982). There is one control run, so the control's own variation between seeds is unknown.

6. Why the joint runs generalize worse

6.1 Rare tokens' embedding vectors

Mean norm of E[t], by token frequency (rare / common / frequent):

run	asked-about tokens	never-asked tokens
LM-only, seed 0 (seed-0 split)	4.68 / 1.65 / 1.44	4.77 / 1.68 / 1.46
joint, seed 0	1.53 / 1.44 / 1.33	3.95 / 1.75 / 1.41
joint, seed 1	1.66 / 1.50 / 1.37	4.33 / 1.80 / 1.42
control, seed 0	4.11 / 1.70 / 1.41	4.13 / 1.75 / 1.43

Follow by token frequency (rare / common / frequent; at most 64 tokens per group):

run	asked-about tokens	never-asked tokens
joint, seed 0	0.85 / 0.85 / 0.85	0.34 / 0.79 / 0.83
joint, seed 1	0.73 / 0.75 / 0.74	0.27 / 0.68 / 0.73
control, seed 0	0.62 / 0.90 / 0.90	0.62 / 0.90 / 0.90

Seed 0 has 248 rare asked-about and 97 rare never-asked tokens. The rare embedding vectors are nearly parallel (mean pairwise cosine 0.998 for the rare never-asked tokens of joint seed 0 and of the control), so they test essentially one direction, not dozens.

Observations. - Next-token prediction alone gives rare tokens' vectors about three times the norm of the others' (LM-only). - In the joint runs, rare asked-about tokens end with ordinary norms; rare never-asked tokens end with large norms; and follow on the rare never-asked tokens is low (0.34, 0.27). - In the control, rare tokens end with large norms in both groups, and follow is the same for both groups (0.62).

6.2 Rescaling test

Each vector is rescaled to a given norm, keeping its direction, and the answers about it are scored by the rescaling score: $1 - (\text{mean over the vectors of the squared distance between the 128 answers and the vector}) / (\text{mean squared distance of all 1,024 never-asked vectors from their coordinate means})$.

never-asked vectors	joint, seed 0	joint, seed 1	control, seed 0
rare, as they are	-1.34	-3.47	0.99
rare, rescaled to the norm of the frequent ones	0.96	0.90	0.97
frequent, as they are	0.94	0.91	0.97
frequent, rescaled to the norm of the rare ones	-1.67	-2.75	-1.45

Observations. - In the joint runs, the rare never-asked vectors are answered badly at their own norm and well at the frequent tokens' norm. - In the control, the rare never-asked vectors are answered well at their own norm. - In all three runs, frequent vectors rescaled to the rare tokens' norm are answered badly.

So whether a vector with a large norm is answered well depends on whether asked-about vectors of similar norm and direction existed in training. In the control, the rare vectors (norms about 4, all nearly in one direction) include asked-about tokens: 348 asked-about vectors have norm above 3. In the joint runs, no asked-about vector has norm above 1.87 (seed 0: 1.83), while never-asked vectors reach 4.06 (seed 0) and 4.45 (seed 1). Frequent vectors rescaled to norm about 4 along their own directions are unlike anything asked about in any run.

6.3 Interpretation and an open alternative

Interpretation. In the joint runs, the self-report task changes the embedding vectors of the asked-about tokens: among other changes, it keeps the norms of the rare ones small. The answers are then fitted to asked-about vectors whose distribution differs from that of the never-asked vectors, and they do not carry over. In the control, asked-about and never-asked vectors come from the same distribution, and the answers carry over completely. Letting the self-report task change the embedding vectors costs less next-token loss than forbidding it: +0.10 nats per token (joint, seed 0) against +0.14 (control), one run each.

What the control establishes: the gradient of the self-report loss on the token embedding vectors is what keeps the norms of the rare asked-about vectors small. What it does not establish is how. Two mechanisms fit: 1. The gradient moves those vectors toward a region where the answers are accurate. 2. An effect of Adam: Adam divides each parameter's step by a running average of the size of its recent gradients. A rarely used embedding vector with a small but consistent gradient from next-token prediction therefore takes full-size steps and its norm grows. An additional, noisy gradient on the same vector (here from the self-report task) enlarges that average and slows the growth, whatever its direction.

7. Response to the norm of $E[t]$, and LayerNorm

The norm of $E[t]$ can change along the direction of $E[t]$ itself. Gains ($u^T J u$) along that direction and, on average, along the 127 directions perpendicular to it:

run	gain along $E[t]$: asked-about / never-asked	gain perpendicular to $E[t]$: asked-about / never-asked
joint, seed 0	0.66 / 0.49	0.85 / 0.75
joint, seed 1	0.57 / 0.42	0.75 / 0.67
control, seed 0	0.75 / 0.75	0.87 / 0.87

In every run, the answers respond less to a change of the norm of $E[t]$ than to a change of its direction.

An architectural fact explains part of this. The input of the first block at position 2 is $v = E[t] + P[2]$. Its LayerNorm first subtracts the mean of v 's coordinates, giving the centred vector c , and then divides by the standard deviation, which is proportional to $|c|$. Multiplying c by a positive number therefore changes nothing that the first block computes. So a change of $E[t]$ along $u = c/|c|$ is invisible to the first block; later blocks receive it only through the internal state, which carries it forward unchanged. $P[2]$ has norm 1.1 to 1.4, comparable to $|E[t]|$, so the direction of $E[t]$ and u are not the same, but they overlap (cosine 0.62 to 0.77). Measured gains along u (64 tokens per group):

run	gain along u : asked-about / never-asked	cosine between $E[t]$ and u : asked-about / never-asked
joint, seed 0	0.005 / 0.006	0.62 / 0.68
joint, seed 1	0.022 / 0.015	0.72 / 0.77
control, seed 0	0.24 / 0.24	0.69 / 0.68

Observation. The joint models barely respond along u (0.005 to 0.022); the control responds partly (0.24). The near-insensitivity along u is therefore not forced by the architecture; it is what training produced in the joint runs. The overlap of u with the direction of $E[t]$ is consistent with the weaker response to the norm of $E[t]$.

A second, exact fact, used as a check of the code: every LayerNorm subtracts the mean, so a change of $E[t]$ along the all-ones direction $(1, \dots, 1)$ reaches the answers only through the output head, which uses the internal state directly, and moves all 128 answers by the same amount, Σw . Measured: all 128 entries of $J \cdot 1$ equal Σw (0.7466 for joint seed 0), with a standard deviation over the 128 answers of about $1.2 \cdot 10^{-7}$, which is single-precision rounding.

8. Editing an embedding vector inside the model

Purpose: to confirm that changing a stored embedding vector changes both the answers about that token and the model's use of the token in text, and to measure by how much. Some response of both kinds is guaranteed, since the edited vector is both an input and an output weight; the measurement is of size.

For each of the 20 never-asked tokens that occur most often in 20×32 validation windows (among them " it", " he", " in", " said", " she", " his", " day"), I added to its stored embedding vector a change of 10% of its norm in a random direction, and compared the model's outputs with and without the change.

	joint, seed 0	joint, seed 1	control, seed 0
finite-change follow of the answers about the edited token: range over the 20 tokens (mean)	0.73 to 0.90 (0.82)	0.61 to 0.90 (0.72)	0.72 to 0.91 (0.83)

	joint, seed 0	joint, seed 1	control, seed 0
change of the next-token distribution where the edited token is the current input, divided by the change at positions with no occurrence of the token so far (ratio of the means over the 20 tokens)	32	34	31
the same, with all positions of all 20 tokens pooled	27	30	28
tokens for which the first change is larger	19 of 20	20 of 20	19 of 20

The change of the next-token distribution is the Kullback–Leibler divergence of the edited from the original distribution. It is not zero at positions without the token, because the edited vector is also used to score that token as a possible next token at every position.

9. A linear map from the LM-only model’s internal state

To see how much information about $E[t]$ the LM-only model’s internal state already holds: a linear map from the internal state of the LM-only model at position 2 (input [QUERY, COORD_0, t]; the QUERY and COORD vectors of this model are untrained) to $E[t]$, fitted by ridge regression on the asked-about tokens, gives centred R^2 0.834 on the never-asked tokens (seed-0 split; 0.803 with the seed-1 split). This map has a separate weight vector for each of the 128 coordinates; the models’ output head is a single weight vector, and the coordinate asked about enters only through the COORD_i symbol. Takeaway: a simple readout of $E[t]$ from an LM-only model’s internal state generalizes to never-asked tokens better than the joint models’ answers (0.680, 0.504) and worse than the control’s (0.982).

10. What this does and does not show

- A small language model can be trained to answer questions about its own embedding coordinates with answers that respond to the current embedding vector (follow 0.87 in the control) and are as accurate for tokens never asked about as for tokens asked about (centred R^2 0.982), at a cost of 0.14 nats per token in next-token prediction.
- If the self-report task may change the embedding vectors it is asked about, it does, and its answers then generalize much worse to the other tokens.
- The answers are never exact: other movement is 0.35 to 0.48 in every run. Whether LayerNorm is why the answers are less exact than in the toy model is a hypothesis only: the toy model also had random embedding vectors, no next-token prediction, and a different training budget.
- The embedding vector of t is the input at t’s position, so the model is reporting a value present in its own input. Weights that are not inputs (for example the feed-forward matrices) remain untested (PLAN.md, roadmap).
- Not tested: training only the self-report path on top of a fixed LM-only model; other values of the loss coefficient λ ; other model sizes; more seeds (one control run and one LM-only run).

11. Checks

- A separate Claude instance reviewed the code before the runs, another a draft of this report, and a third the terms used in it; their findings were checked and the fixes are recorded in NOTES.md.
- 15 tests (tests/test_lm.py), including checks of every measure on hand-made functions with known answers, the all-ones fact of section 7, and that the control's self-report loss gives no gradient to the token embedding vectors.
- Scripts: train.py; measure.py (sections 5 and 8; files results/measure_json); analyze_review.py (follow by frequency in 6.1, 6.2, gains along u in 7, section 9; files results/review_checks_json); norms_by_frequency.py (norm tables in 6.1; results/norms_by_frequency.json); summarize.py.